

We burned 48 GPU hours training on 96 million tokens

And the loss curve told us for hours. What a year of running a thousand-run experiment grid on other people's GPUs teaches you about failure, and about the question underneath every one of them: who owns the decision.

Not a production service. It behaves like one

878

archived training runs behind one research project

4

compute platforms in a year: a desk GPU, a lab machine, RunPod, Modal

0

of the expensive failures were modelling bugs

Why the operational character is the same

A job that dies at hour twenty is expensive. A job that succeeds while computing the wrong thing is worse, because nothing pages you. Small enough that every failure is legible, large enough that each one costs real money.

01

The run that looked fine

Twenty four hours, in four beats

Hour 0 The data preparation job starts writing shards to a shared network volume.

Hour 0 Two training containers launch against the same volume. Neither waits.

Hour 6 Validation loss starts climbing. I read it as evaluation noise and leave it.

Hour 24 Run ends. Train loss 0.95. Validation loss 5.6.

48

H100 hours, gone

Two containers, one full day each

What actually happened

Both containers trained on shard zero alone

96 million tokens instead of the full corpus. About thirty epochs of the same data in twenty four hours. Train loss 0.95 against validation loss 5.6 is not a model that learned. It is a model that memorised.

The cause, in one sentence

A container only sees files another container has committed after it explicitly reloads the volume. Neither training container ever reloaded, so both saw the corpus exactly as it looked the instant they started.

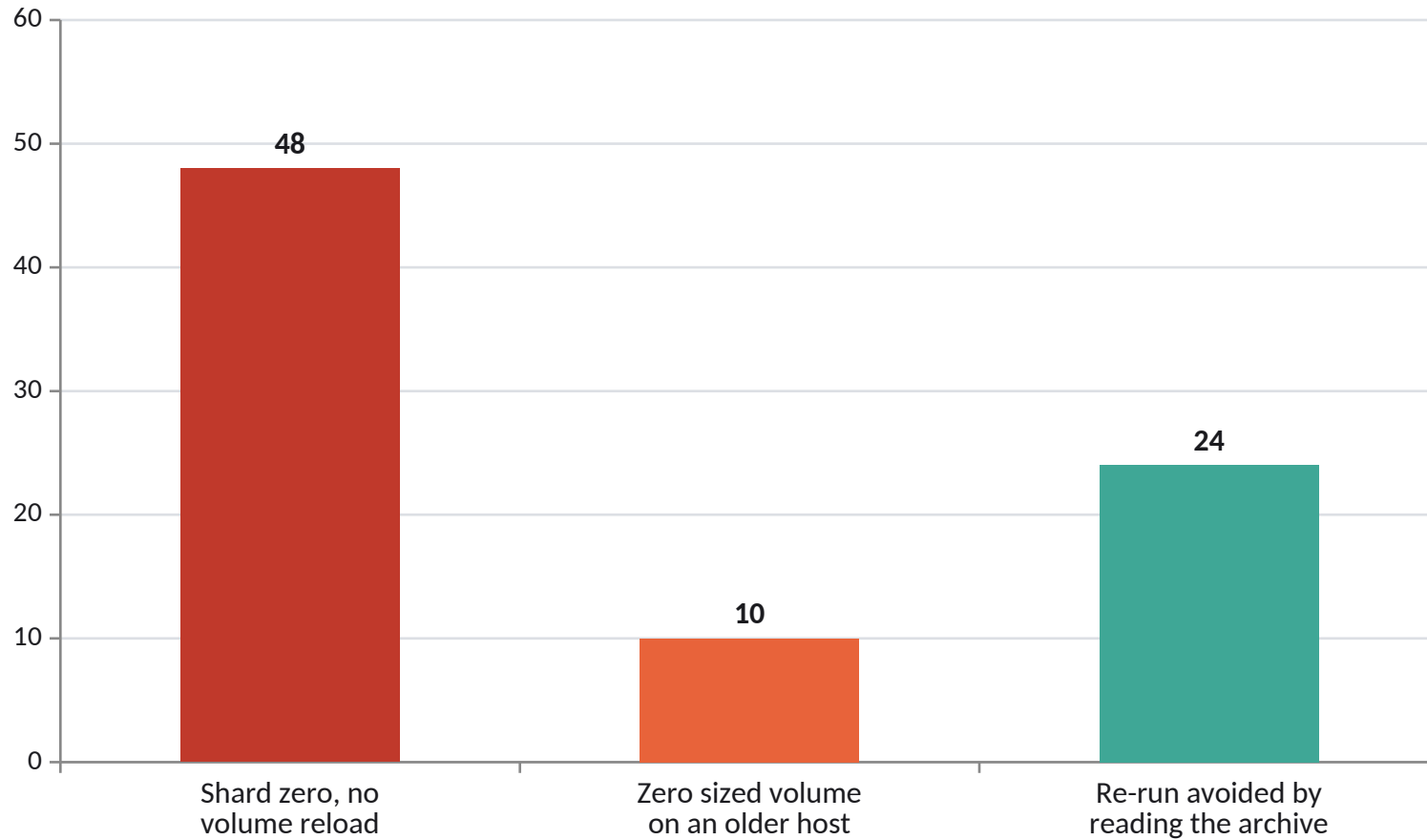
Nobody called it a cache. It is a cache.

A producer and a consumer sharing a network volume is a distributed systems problem, not a filesystem operation. I had been treating it as a directory.

02

It was never one bug

What each one cost, in GPU hours



The green one is a saving

An experiment that already existed, sitting in a results archive while the repository said it had never been run. One download instead of a GPU day.

The cheapest to fix

Was also the one with no error, no alert and no stack trace. It was a question of who owned the result, not of what the code did.

Four more, and none of them were the model

1

The foreground launch

The platform's default mode ties the remote job's life to my laptop's connection. A local DNS blip tore down four concurrently running jobs. They survived only because the scripts checkpoint.

2

A twenty four hour function cap

Discovered while planning a six day run. Fix is a resume chain: run under a timeout, checkpoint on the termination signal, respawn on the timeout exit code.

3

Two workspaces, one volume name

The same volume name existed in two workspaces with different contents. A listing from the wrong one looked exactly like catastrophic data loss, for about ten very bad minutes.

4

A zero sized persistent volume

A pod created with no persistent storage attached on an earlier platform. Ten hours of work, nothing on disk at the end of it.

The common shape

Every expensive failure was a question about what was responsible for something, answered wrong.

Shared state

Who guarantees the input is complete before a consumer reads it? I assumed the filesystem did. It does not.

Lifetime

What keeps the remote work alive, and for how long? I assumed my terminal was incidental. It was load bearing.

Ownership of results

Who is responsible for a finished result reaching the place people read? Nobody was, so it did not.

Not one of them needed a better model. All three needed somebody to have decided who was responsible.

03

What I do differently now

Four practices

1 Cheap screens before spend

Compile it, run one tiny pass end to end on CPU, measure real throughput, then choose a budget. This has caught more of my expensive mistakes than reading the code ever did.

2 Consumers assert, they do not trust

A training job now refuses to start unless the token count it can see matches the token count it was promised. The check is three lines and would have saved 48 hours.

3 Detach by default, checkpoint always

Assume the control plane will disconnect, because it will. Every failure in this talk was survivable in proportion to how recently the job had checkpointed.

4 List the archive before you run

The cheapest experiment is the one you already ran. Checking costs one API call and has twice now returned a result I was about to pay to recompute.

And what each one costs

A runner, or one off scripts

I built a generic runner that passes paths through environment variables. It made the scripts portable across four hosts with no edits.

The cost is a subprocess boundary that hides real errors, and two of the failures in this talk were harder to diagnose because of it.

Checkpoint often, or run fast

Frequent checkpointing costs throughput on every single run, forever.

It has also saved every run that was interrupted, which by now is most of them. I stopped treating this as a tuning parameter.

Fix the script, or fix the runner

Several failures came from scripts making assumptions about their own working directory.

I fixed them at the runner level rather than editing scripts with an archived result history, knowingly leaving the root cause in place to keep old results comparable.

04

The failure with no stack trace

The decision my whole project turns on

Quantisation: storing every number in the model with fewer digits so it runs on cheaper hardware.

A cost decision

Memory, throughput, the hardware you have to buy.

A platform decision

The serving stack has to expose it, and own the rollback.

A quality decision

The served model changes numerically, with no version bump.

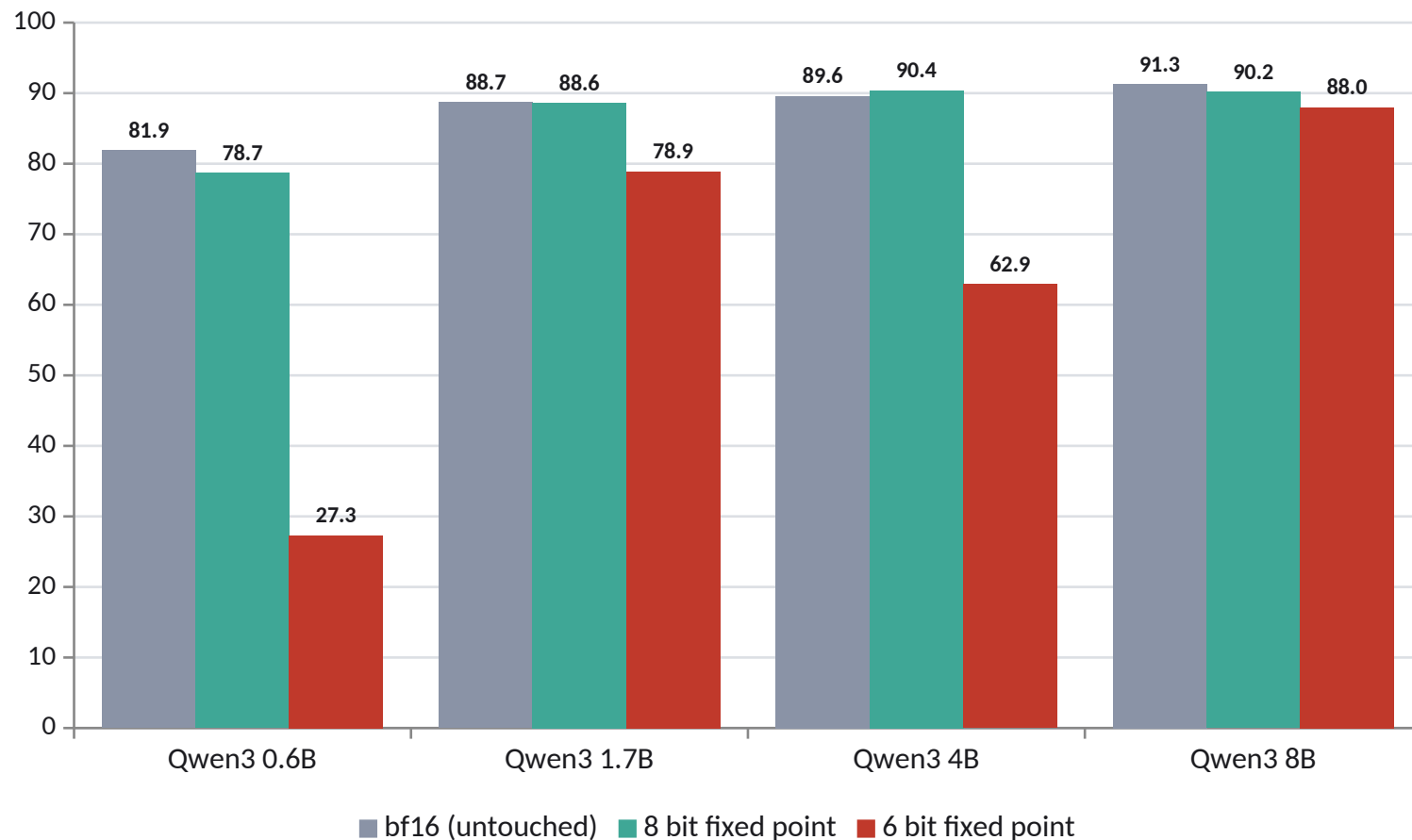
A hardware decision

Weights and activations want different widths. Silicon has to agree.

In most organisations, nobody owns all four.

So it either never happens, or it happens without a validation story. Same failure mode as the result that sat in an archive nobody owned.

What the evidence actually looks like



Eight bits is free

Function calling accuracy holds within about a point of the original at every size, with no retraining and no calibration set.

Six bits is not one answer

Free at 8B. Fatal at 0.6B. One model would have produced confident, wrong guidance in either direction.

Two failures that look like success

It scored 100 percent because it stopped answering

One benchmark category scores a model on correctly declining to call a tool. A model that had stopped calling tools entirely, on 98.8 percent of prompts, scored a perfect 100 there. Averaged across categories it would have been published at 28.7 percent: mediocre, not dead.

Broken at sixteen bits

Two projection matrices in each block are fed by no normalisation layer. Quantised without a scale, validation loss went from 2.53 to 7.73, at sixteen bits, where the grid step is three in a hundred thousand. A few hundred outlier weights, clipped.

Every capability metric needs a liveness metric beside it.

For tool calling that is the call rate. For your system it is whatever number separates correctly did nothing from did nothing.

The part I cannot answer, and you can

Who signs off when the served model changes numerically but not by version number?

What evidence does your team require, and has anyone ever actually rejected a change on it?

Should the platform expose precision as a knob at all, or is that a decision that belongs upstream?

How would you find out if a quantised model got worse in a way your current metrics do not show?

I have measured the technical side to death. How other teams organise this decision, I have almost no visibility into.

What to take away

1

Shared state between jobs is a distributed systems problem

not a filesystem operation. Make the consumer assert, not trust.

2

A cheap screen catches more than careful review

a CPU smoke test and a throughput probe, every time, before anything paid starts.

3

The failures that cost the most had no owner

not no alert. Ask who owns the decision before you ask what the code did.